

《软件技术基础课程设计》

实验项目：2-1 团队数据库

院　　系：计算机科学与技术系

学　　期：2024 年第 1 学期

班　　级：计算机科学与技术 2 班

学　　号：226001624

姓　　名：顾凯琪

指导教师：叶绪伦

日期：2024 年 12 月 16 日

实验二名称：团队综合编程实验（报告+答辩/4-5 人）

任务二-1（数据库连接和数据存储）
可采用合适的 Visual Studio 版本开发
最新版本的 VS 对 C++的可视化支持已做调整，可采用 C#
可采用其他语言/可视化工具/数据库等完成指定功能，请在解决方案中说明

数据库连接和数据存储		
学会连接数据库，并编写数据库应用程序		
学号	姓名	任务分配
组长	赖诗亮	后端代码编写　33%
成员 1	崔智英	前端代码编写　33%
成员 2	顾凯琪	数据库代码　33%

【评分】：1、基础任务完成及质量；2、过程分析/讨论/文档质量；3、扩展深度/质量
【拓展部分】:（1）指在题目的基础上，自行加强设计和功能并完成，可不扩展、1 题扩展或多题扩展；（2）扩展部分是评分标准之一，占一定分值；（3）请说明扩展的内容/代码说明/运行结果。
【代码】：代码放文档中，无需另交项目/代码文件

一、实验内容

建立一个简单的数据库/几张表，编程实现增删改查的功能
（一）、手动连接数据库（与下一个的程序连接数据库是独立的 2 个部分）
（二）编程中使用数据库。

二、实验过程

（一）、手动连接数据库（与下一个的程序连接数据库是独立的 2 个部分）
（1）使用 flask 框架进行后端管理

```
app = Flask(__name__, static_folder='unimp')
# 配置数据库的地址
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql://root:admin123456@localhost:3306/database'
# 跟踪数据库的修改 --> 不建议开启 未来的版本中会移除
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = secrets.token_hex(16)
# 启用 CORS，并允许所有来源的请求访问你的 Flask 应用中的所有路径
CORS(app, resources={r"/*": {"origins": "*"}})

# 数据库初始化
db.init_app(app)
```

图 1 后端数据库连接设置

```
import axios from 'axios';

const apiClient = axios.create({
  baseURL: 'http://localhost:5000', // 你的 Flask 应用的地址
  withCredentials: true, // 这个设置取决于你的后端是否需要 cookies
  headers: {
    Accept: 'application/json',
    'Content-Type': 'application/json'
  }
});

export default apiClient;
```

图 2 前端数据库连接设置

（2）启动程序连接数据库

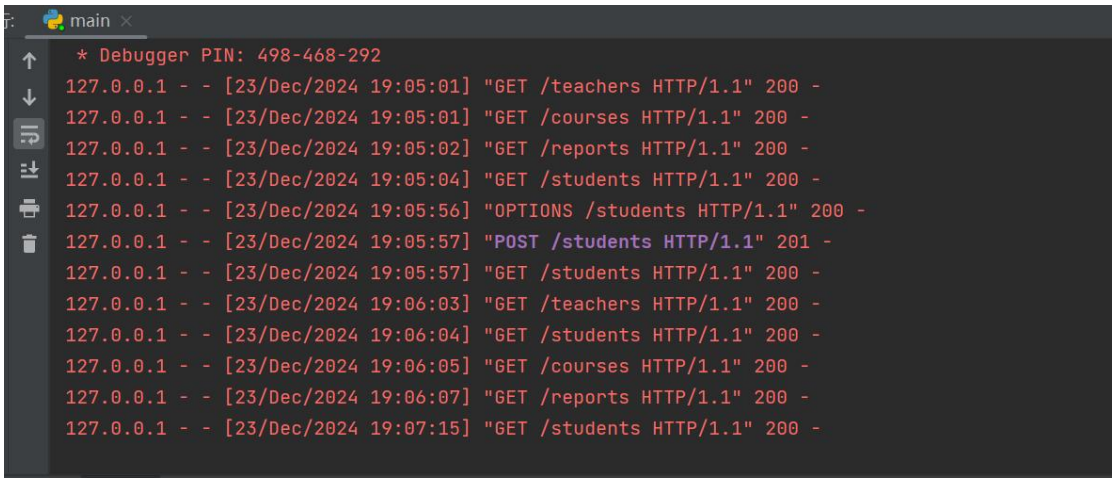


图 3 成功连接数据库

（二）编程中使用数据库

（1）数据库中先建立四张表，用以数据的定义与存储

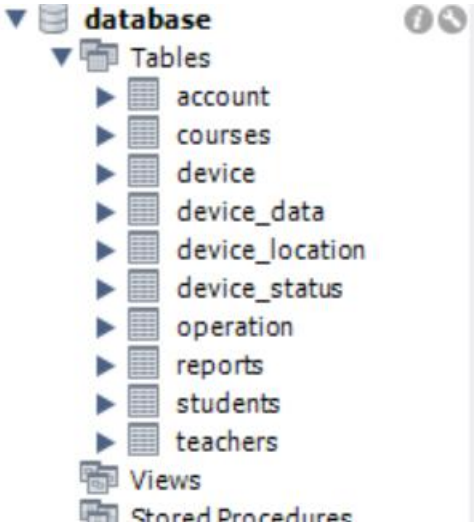


图 4 数据库中表的定义

（2）编写相应的前端页面：学生管理页面



图 5 学生管理

（3）编写相应的前端页面：教师管理页面



图 6 教师管理

（4）编写相应的前端页面：课程管理页面



图 7 课程管理

（5）编写相应的前端页面：成绩管理页面

学生 | 老师 | 课程 | 成绩

成绩管理

添加新记录

学生编号 (Sno)	教师编号 (Tno)	课程编号 (Cno)	分数 (Score)	操作
S01	T01	C01	83	编辑 删除
S01	T03	C03	85	编辑 删除
S02	T01	C01	75	编辑 删除
S02	T01	C08	83	编辑 删除
S02	T02	C02	45	编辑 删除
S02	T02	C09	77	编辑 删除
S02	T03	C03		编辑 删除
S02	T04	C04		编辑 删除
S02	T04	C06	83	编辑 删除
S02	T05	C05	70	编辑 删除

图 8 成绩管理

（6）学生管理添加记录

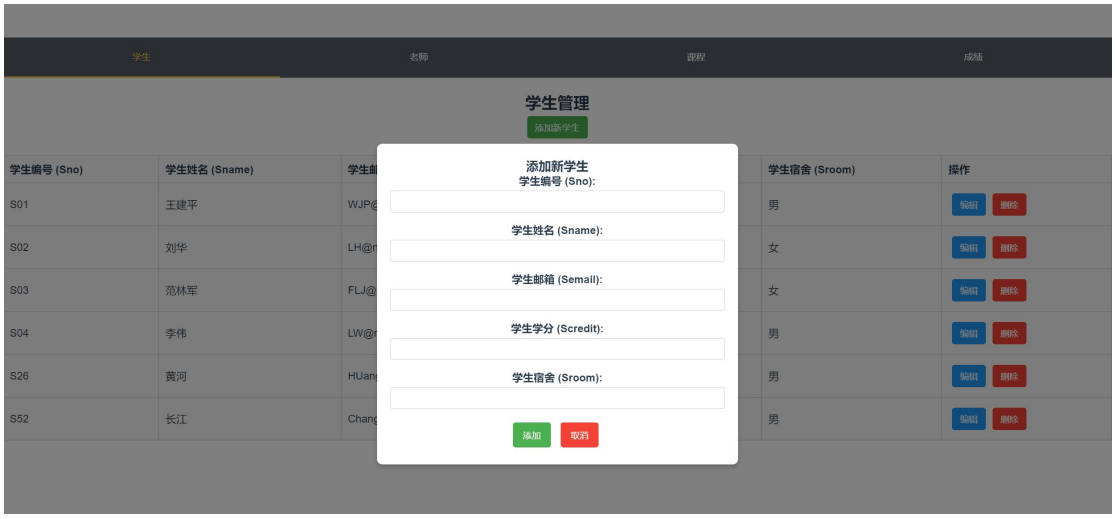


图 10 学生添加

学生 | 老师 | 课程 | 成绩

学生管理

添加新学生

学生编号 (Sno)	学生姓名 (Sname)	学生邮箱 (Semail)	学生学分 (Scredit)	学生宿舍 (Sroom)	操作
0	赖诗亮	222666@qq	200	7-216	编辑 删除
S01	王建平	WJP@nbu.edu.cn	23.1	男	编辑 删除
S02	刘华	LH@nbu.edu.cn	24.6	女	编辑 删除
S03	范林军	FLJ@nbu.edu.cn	16.6	女	编辑 删除
S04	李伟	LW@nbu.edu.cn	15.8	男	编辑 删除
S26	黄河	HUanghe@nbu.edu.cn	13.4	男	编辑 删除
S52	长江	Changjiang@nbu.edu.cn	12.4	男	编辑 删除

图 11 页面更新

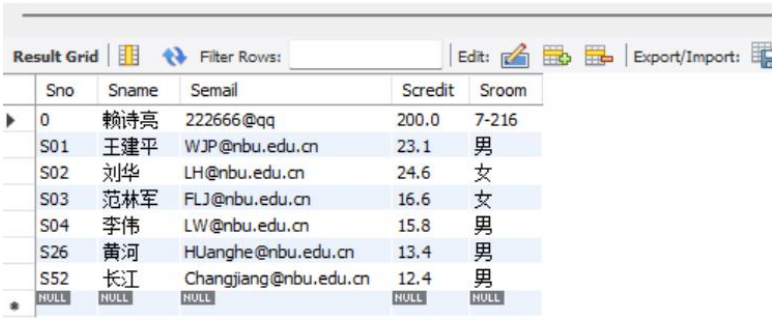


图 12 数据库同步更新

（7）教师管理添加记录

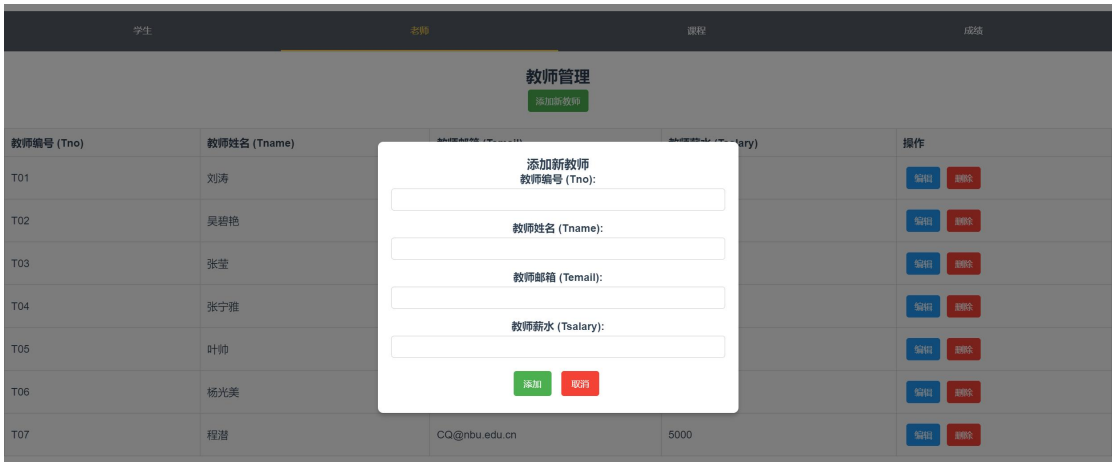


图 13 教师添加

（8）课程管理添加记录

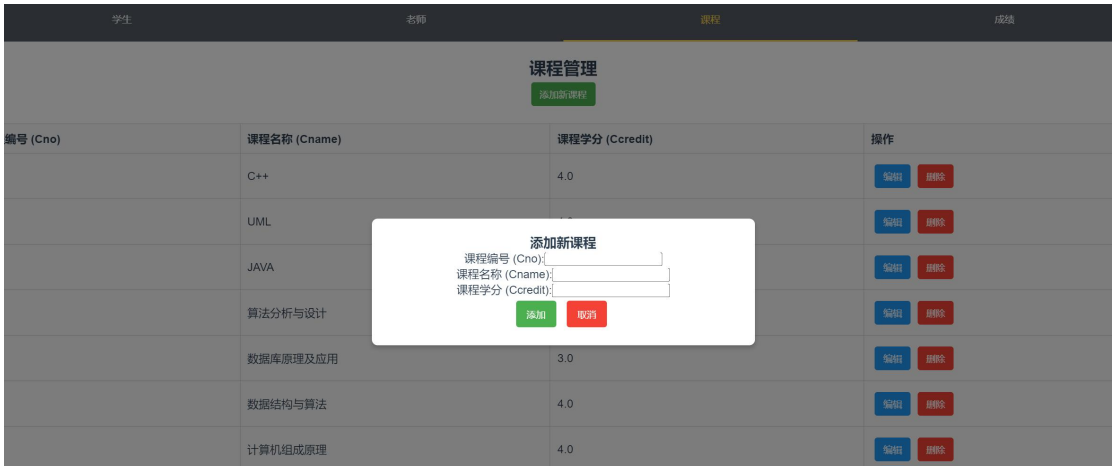


图 14 课程添加

（9）教师管理添加记录

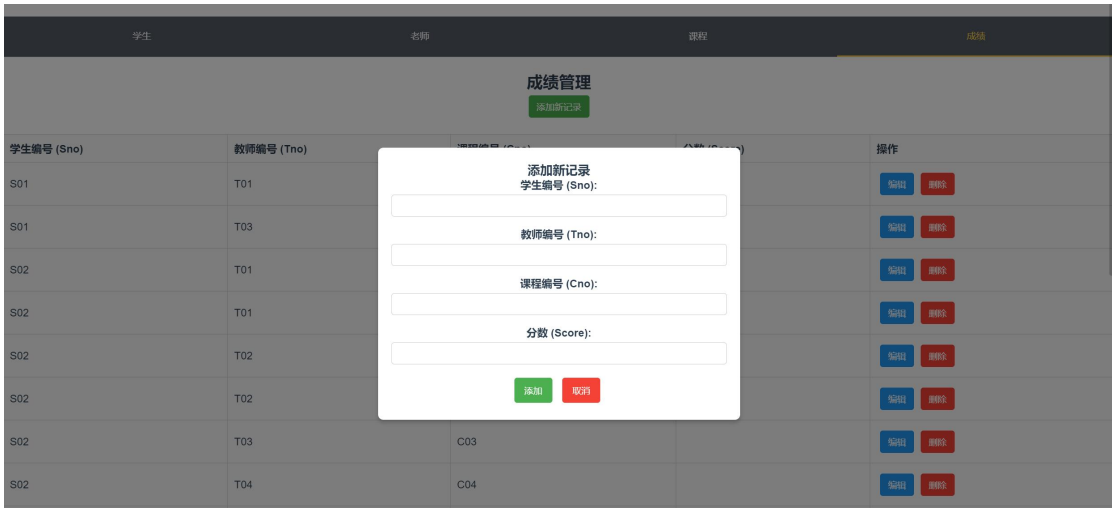


图 15 成绩记录添加

（10）学生信息修改

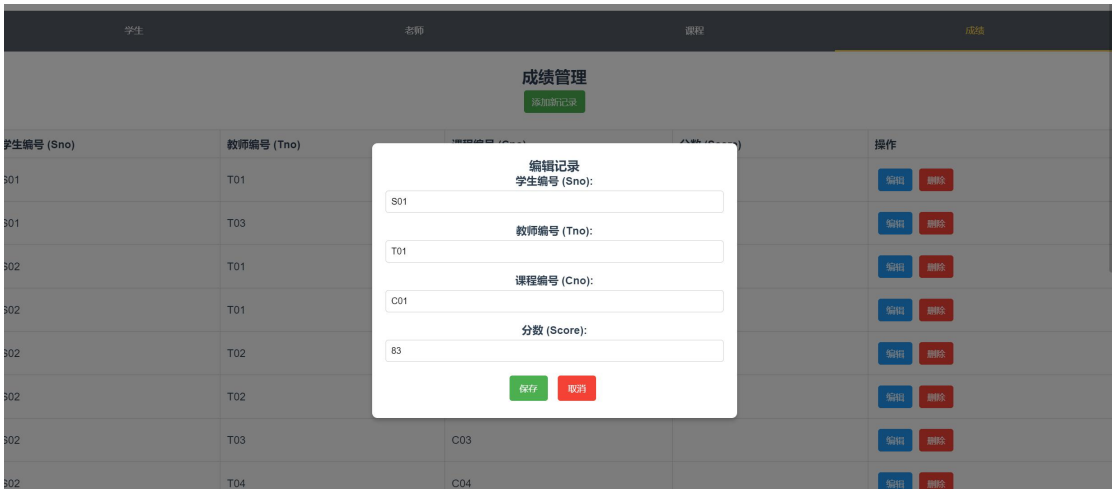


图 13 学生修改

三、代码实现

（1）sql 代码

```
-- 如果存在 Reports 表则删除
DROP TABLE IF EXISTS Reports;
-- 如果存在 Students 表则删除
DROP TABLE IF EXISTS Students;
-- 创建 Students 表，定义列及相应完整性约束条件
CREATE TABLE Students (
    Sno VARCHAR(6),  -- 学号，定义为长度为6的字符串类型
    Sname VARCHAR(20) NOT NULL,  -- 学生姓名，不能为空值
    Semail VARCHAR(50),
```

```
    Scredit DECIMAL(5, 1),  -- 学分，设置精度为总长度5位，小数位1位
    Sroom VARCHAR(5),
    -- 定义主键约束，将 Sno 字段设为主键
    CONSTRAINT PK_Stu PRIMARY KEY (Sno),
    -- 定义检查约束，确保学分 Scredit 大于等于0
    CONSTRAINT CK_Student_Scredit CHECK (Scredit >= 0)
);
-- 如果存在 Teachers 表则删除
DROP TABLE IF EXISTS Teachers;
-- 创建 Teachers 表，定义列及相应完整性约束条件
CREATE TABLE Teachers (
    Tno VARCHAR(6),  -- 教师编号，定义为长度为6的字符串类型
    Tname VARCHAR(20) NOT NULL,  -- 教师姓名，不能为空值
    Temail VARCHAR(50),
    Tsalary DECIMAL(5, 1),  -- 教师工资，设置精度为总长度5位，小数位1位
    -- 定义主键约束，将 Tno 字段设为主键
    CONSTRAINT PK_Tea PRIMARY KEY (Tno)
);
-- 如果存在 Courses 表则删除
DROP TABLE IF EXISTS Courses;
-- 创建 Courses 表，定义列及相应完整性约束条件
CREATE TABLE Courses (
    Cno VARCHAR(6),  -- 课程编号，定义为长度为6的字符串类型
    Cname VARCHAR(40) NOT NULL,  -- 课程名称，不能为空值
    Ccredit DECIMAL(5, 1),  -- 课程学分，设置精度为总长度5位，小数位1位
    -- 定义主键约束，将 Cno 字段设为主键
    CONSTRAINT PK_Cou PRIMARY KEY (Cno)
);
-- 如果存在 Reports 表则删除
DROP TABLE IF EXISTS Reports;
-- 创建 Reports 表，用于存储学生、教师、课程之间的成绩关联信息，定义列及相应完整性约束条件
CREATE TABLE Reports (
    Sno VARCHAR(6),
    Tno VARCHAR(6),
    Cno VARCHAR(6),
    Score DECIMAL(5, 1),
    -- 定义复合主键约束，由 Sno、Tno、Cno 三个字段共同构成主键
    CONSTRAINT PK_Rep PRIMARY KEY (Sno, Tno, Cno),
    -- 定义外键约束，关联到 Students 表的 Sno 字段
    CONSTRAINT FK_Stu_Rep FOREIGN KEY (Sno) REFERENCES Students(Sno),
    -- 定义外键约束，关联到 Teachers 表的 Tno 字段
    CONSTRAINT FK_Tea_Rep FOREIGN KEY (Tno) REFERENCES Teachers(Tno),
    -- 定义外键约束，关联到 Courses 表的 Cno 字段
```

```
    CONSTRAINT FK_Cou_Rep FOREIGN KEY (Cno) REFERENCES Courses(Cno)
);
-- 向 Students 表插入多条学生记录
INSERT INTO Students VALUES
    ('S01', '王建平', 'WJP@nbu.edu.cn', 23.1, '男'),
    ('S02', '刘华', 'LH@nbu.edu.cn', 24.6, '女'),
    ('S03', '范林军', 'FLJ@nbu.edu.cn', 16.6, '女'),
    ('S04', '李伟', 'LW@nbu.edu.cn', 15.8, '男'),
    ('S26', '黄河', 'HUanghe@nbu.edu.cn', 13.4, '男'),
    ('S52', '长江', 'Changjiang@nbu.edu.cn', 12.4, '男');
-- 向 Teachers 表插入多条教师记录
INSERT INTO Teachers VALUES
    ('T01', '刘涛', 'LT@nbu.edu.cn', 4300),
    ('T02', '吴碧艳', 'WBY@nbu.edu.cn', 2500),
    ('T03', '张莹', 'ZY@nbu.edu.cn', 3000),
    ('T04', '张宁雅', 'ZNY@nbu.edu.cn', 5500),
    ('T05', '叶帅', 'YS@nbu.edu.cn', 3800),
    ('T06', '杨光美', 'YGM@nbu.edu.cn', 3500),
    ('T07', '程潜', 'CQ@nbu.edu.cn', 5000);
-- 向 Courses 表插入多条课程记录
INSERT INTO Courses VALUES
    ('C01', 'C++', 4),
    ('C02', 'UML', 4),
    ('C03', 'JAVA', 3),
    ('C04', '算法分析与设计', 3),
    ('C05', '数据库原理及应用', 3),
    ('C06', '数据结构与算法', 4),
    ('C07', '计算机组成原理', 4),
    ('C08', '英语', 6),
    ('C09', '数字生活', 2),
    ('C10', '音乐鉴赏', 2),
    ('C11', '体育 1', 2);
-- 向 Reports 表插入多条成绩记录，部分成绩字段可为 NULL
INSERT INTO Reports VALUES
    ('S01', 'T01', 'C01', 83),
    ('S01', 'T03', 'C03', 85),
    ('S02', 'T01', 'C01', 75),
    ('S02', 'T02', 'C02', 45),
    ('S02', 'T03', 'C03', NULL),
    ('S02', 'T04', 'C04', NULL),
    ('S02', 'T05', 'C05', 70),
    ('S02', 'T04', 'C06', 83),
    ('S02', 'T05', 'C07', 90),
    ('S02', 'T01', 'C08', 83),
```

```
    ('S02', 'T02', 'C09', 77),
    ('S02', 'T07', 'C10', 83),
    ('S02', 'T06', 'C11', 88),
    ('S03', 'T01', 'C08', 63),
    ('S03', 'T02', 'C02', 93),
    ('S03', 'T01', 'C01', 78),
    ('S04', 'T06', 'C06', 89),
    ('S04', 'T05', 'C05', 93),
    ('S26', 'T07', 'C10', 45),
    ('S26', 'T04', 'C04', 86),
    ('S52', 'T07', 'C10', 91),
    ('S52', 'T06', 'C11', 90),
    ('S52', 'T05', 'C05', NULL),
    ('S52', 'T01', 'C08', 64),
    ('S52', 'T02', 'C09', 81);
```

（2）后端代码主文件

```
from flask import Flask, render_template, request, jsonify, redirect, u
rl_for, flash
from extension import db
from models import Students, Teachers, Courses, Reports
from datetime import datetime
import secrets
from flask_cors import CORS
from api.courses import courses_api
from api.reports import reports_api
from api.students import students_api
from api.teachers import teachers_api
app = Flask(__name__, static_folder='unimp')
# 配置数据库的地址
app.config['SQLALCHEMY_DATABASE_URI'] = 'mysql://root:admin123456@local
host:3306/database'
# 跟踪数据库的修改 --> 不建议开启 未来的版本中会移除
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = secrets.token_hex(16)
# 启用 CORS，并允许所有来源的请求访问你的 Flask 应用中的所有路径
# 启用 CORS 并允许携带凭据
CORS(app, supports_credentials=True)
# 数据库初始化
db.init_app(app)
# 注册各个 API 蓝图
app.register_blueprint(courses_api)
app.register_blueprint(reports_api)
app.register_blueprint(students_api)
```

```
app.register_blueprint(teachers_api)
if __name__ == "__main__":
    with app.app_context():
        db.create_all()  # 创建所有表
    app.run(debug=True)
```

（3）后端数据模型定义

```
from extension import db
from sqlalchemy import DECIMAL, VARCHAR, ForeignKey
class Students(db.Model):
    __tablename__ = 'Students'
    Sno = db.Column(VARCHAR(6), primary_key=True)
    Sname = db.Column(VARCHAR(20), nullable=False)
    Semail = db.Column(VARCHAR(50))
    Scredit = db.Column(DECIMAL(5, 1), nullable=False)
    Sroom = db.Column(VARCHAR(5))
    __table_args__ = (
        db.CheckConstraint('Scredit >= 0', name='CK_Student_Scredit'),
    )
class Teachers(db.Model):
    __tablename__ = 'Teachers'
    Tno = db.Column(VARCHAR(6), primary_key=True)
    Tname = db.Column(VARCHAR(20), nullable=False)
    Temail = db.Column(VARCHAR(50))
    Tsalary = db.Column(DECIMAL(5, 1))
class Courses(db.Model):
    __tablename__ = 'Courses'
    Cno = db.Column(VARCHAR(6), primary_key=True)
    Cname = db.Column(VARCHAR(40), nullable=False)
    Ccredit = db.Column(DECIMAL(5, 1))
class Reports(db.Model):
    __tablename__ = 'Reports'
    Sno = db.Column(VARCHAR(6), ForeignKey('Students.Sno'), primary_key
=True)
    Tno = db.Column(VARCHAR(6), ForeignKey('Teachers.Tno'), primary_key
=True)
    Cno = db.Column(VARCHAR(6), ForeignKey('Courses.Cno'), primary_key=
True)
    Score = db.Column(DECIMAL(5, 1))
```

（4）学生管理 api

```
from flask import Blueprint, jsonify, request
from models import Students, db
students_api = Blueprint('students_api', __name__)
```

```python
# 获取所有学生
@students_api.route('/students', methods=['GET'])
def get_students():
    try:
        students = Students.query.all()
        students_list = [
            {
                "Sno": student.Sno,
                "Sname": student.Sname,
                "Semail": student.Semail,
                "Scredit": float(student.Scredit),  # 确保返回为浮点数
                "Sroom": student.Sroom
            }
            for student in students
        ]
        return jsonify(students_list), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
# 添加新学生
@students_api.route('/students', methods=['POST'])
def add_student():
    try:
        data = request.json
        # 校验字段
        if not data.get('Sno') or len(data['Sno']) > 6:
            return jsonify({"error": "学生编号无效"}), 400
        if not data.get('Sname') or len(data['Sname']) > 20:
            return jsonify({"error": "学生姓名无效"}), 400
        if data.get('Semail') and len(data['Semail']) > 50:
            return jsonify({"error": "学生邮箱无效"}), 400
        if data.get('Scredit') is None or data['Scredit'] < 0:
            return jsonify({"error": "学生学分无效"}), 400
        new_student = Students(
            Sno=data['Sno'],
            Sname=data['Sname'],
            Semail=data.get('Semail'),
            Scredit=data['Scredit'],
            Sroom=data.get('Sroom')
        )
        db.session.add(new_student)
        db.session.commit()
        return jsonify({"message": "学生添加成功"}), 201
    except Exception as e:
        db.session.rollback()
```

```
        return jsonify({"error": str(e)}), 400
# 更新学生信息
@students_api.route('/students/<Sno>', methods=['PUT'])
def update_student(Sno):
    try:
        data = request.json
        student = Students.query.get(Sno)
        if not student:
            return jsonify({"error": "学生不存在"}), 404
        # 校验字段
        if data.get('Sname') and len(data['Sname']) > 20:
            return jsonify({"error": "学生姓名无效"}), 400
        if data.get('Semail') and len(data['Semail']) > 50:
            return jsonify({"error": "学生邮箱无效"}), 400
        if data.get('Scredit') is not None and data['Scredit'] < 0:
            return jsonify({"error": "学生学分无效"}), 400
        student.Sname = data.get('Sname', student.Sname)
        student.Semail = data.get('Semail', student.Semail)
        student.Scredit = data.get('Scredit', student.Scredit)
        student.Sroom = data.get('Sroom', student.Sroom)
        db.session.commit()
        return jsonify({"message": "学生信息更新成功"}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
# 删除学生
@students_api.route('/students/<Sno>', methods=['DELETE'])
def delete_student(Sno):
    try:
        student = Students.query.get(Sno)
        if not student:
            return jsonify({"error": "学生不存在"}), 404
        db.session.delete(student)
        db.session.commit()
        return jsonify({"message": "学生删除成功"}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
```

（5）教师管理 api

```
from flask import Blueprint, jsonify, request
from models import Teachers, db
teachers_api = Blueprint('teachers_api', __name__)
# 获取所有教师
```

```python
@teachers_api.route('/teachers', methods=['GET'])
def get_teachers():
    try:
        teachers = Teachers.query.all()
        teachers_list = [
            {
                "Tno": teacher.Tno,
                "Tname": teacher.Tname,
                "Temail": teacher.Temail,
                "Tsalary": float(teacher.Tsalary) if teacher.Tsalary is
 not None else None
            }
            for teacher in teachers
        ]
        return jsonify(teachers_list), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
# 添加新教师
@teachers_api.route('/teachers', methods=['POST'])
def add_teacher():
    try:
        data = request.json
        # 校验字段
        if not data.get('Tno') or len(data['Tno']) > 10:
            return jsonify({"error": "教师编号无效"}), 400
        if not data.get('Tname') or len(data['Tname']) > 50:
            return jsonify({"error": "教师姓名无效"}), 400
        if data.get('Temail') and len(data['Temail']) > 100:
            return jsonify({"error": "教师邮箱无效"}), 400
        if data.get('Tsalary') is not None and data['Tsalary'] < 0:
            return jsonify({"error": "教师薪水无效"}), 400
        new_teacher = Teachers(
            Tno=data['Tno'],
            Tname=data['Tname'],
            Temail=data.get('Temail'),
            Tsalary=data.get('Tsalary')
        )
        db.session.add(new_teacher)
        db.session.commit()
        return jsonify({"message": "教师添加成功"}), 201
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
# 更新教师信息
```

```python
@teachers_api.route('/teachers/<Tno>', methods=['PUT'])
def update_teacher(Tno):
    try:
        data = request.json
        teacher = Teachers.query.get(Tno)
        if not teacher:
            return jsonify({"error": "教师不存在"}), 404
        # 校验字段
        if data.get('Tname') and len(data['Tname']) > 50:
            return jsonify({"error": "教师姓名无效"}), 400
        if data.get('Temail') and len(data['Temail']) > 100:
            return jsonify({"error": "教师邮箱无效"}), 400
        if data.get('Tsalary') is not None and data['Tsalary'] < 0:
            return jsonify({"error": "教师薪水无效"}), 400
        teacher.Tname = data.get('Tname', teacher.Tname)
        teacher.Temail = data.get('Temail', teacher.Temail)
        teacher.Tsalary = data.get('Tsalary', teacher.Tsalary)
        db.session.commit()
        return jsonify({"message": "教师信息更新成功"}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
# 删除教师
@teachers_api.route('/teachers/<Tno>', methods=['DELETE'])
def delete_teacher(Tno):
    try:
        teacher = Teachers.query.get(Tno)
        if not teacher:
            return jsonify({"error": "教师不存在"}), 404
        db.session.delete(teacher)
        db.session.commit()
        return jsonify({"message": "教师删除成功"}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
```

（6）课程管理 api

```python
from flask import Blueprint, jsonify, request
from models import Courses, db
courses_api = Blueprint('courses_api', __name__)
# 获取所有课程
@courses_api.route('/courses', methods=['GET'])
def get_courses():
```

```
    try:
        courses = Courses.query.all()
        courses_list = [
            {
                "Cno": course.Cno,
                "Cname": course.Cname,
                "Ccredit": course.Ccredit
            }
            for course in courses
        ]
        return jsonify(courses_list), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
# 添加新课程
@courses_api.route('/courses', methods=['POST'])
def add_course():
    try:
        data = request.json
        new_course = Courses(
            Cno=data['Cno'],
            Cname=data['Cname'],
            Ccredit=data['Ccredit']
        )
        db.session.add(new_course)
        db.session.commit()
        return jsonify({"message": "课程添加成功"}), 201
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
# 更新课程
@courses_api.route('/courses/<Cno>', methods=['PUT'])
def update_course(Cno):
    try:
        data = request.json
        course = Courses.query.get(Cno)
        if not course:
            return jsonify({"error": "课程不存在"}), 404
        course.Cname = data.get('Cname', course.Cname)
        course.Ccredit = data.get('Ccredit', course.Ccredit)
        db.session.commit()
        return jsonify({"message": "课程更新成功"}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
```

```python
# 删除课程
@courses_api.route('/courses/<Cno>', methods=['DELETE'])
def delete_course(Cno):
    try:
        course = Courses.query.get(Cno)
        if not course:
            return jsonify({"error": "课程不存在"}), 404
        db.session.delete(course)
        db.session.commit()
        return jsonify({"message": "课程删除成功"}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({"error": str(e)}), 400
```

（7）成绩管理 api

```python
from flask import Blueprint, jsonify, request
from models import Reports, db
reports_api = Blueprint('reports_api', __name__)
# 获取所有成绩记录
@reports_api.route('/reports', methods=['GET'])
def get_reports():
    try:
        reports = Reports.query.all()
        reports_list = [
            {
                "id": f"{report.Sno}-{report.Tno}-{report.Cno}",  # 唯一标识符
                "Sno": report.Sno,
                "Tno": report.Tno,
                "Cno": report.Cno,
                "Score": float(report.Score) if report.Score is not None else None
            }
            for report in reports
        ]
        return jsonify(reports_list), 200
    except Exception as e:
        return jsonify({"error": str(e)}), 500
@reports_api.route('/reports', methods=['POST'])
def add_report():
    data = request.get_json()
    new_report = Reports(Sno=data['Sno'], Tno=data['Tno'], Cno=data['Cno'], Score=data['Score'])
    db.session.add(new_report)
```

```
    db.session.commit()
    return jsonify({'message': 'Report added successfully'}), 201
@reports_api.route('/reports/<int:id>', methods=['PUT'])
def update_report(id):
    data = request.get_json()
    report = Reports.query.get_or_404(id)
    report.Sno = data['Sno']
    report.Tno = data['Tno']
    report.Cno = data['Cno']
    report.Score = data['Score']
    db.session.commit()
    return jsonify({'message': 'Report updated successfully'})
@reports_api.route('/reports/<int:id>', methods=['DELETE'])
def delete_report(id):
    report = Reports.query.get_or_404(id)
    db.session.delete(report)
    db.session.commit()
    return jsonify({'message': 'Report deleted successfully'})
```

（8）课程页面组件

```
<template>
  <div>
    <h2>课程管理</h2>
    <button @click="showForm = true" class="add-btn">添加新课程</button>
    <table>
      <thead>
        <tr>
          <th>课程编号 (Cno)</th>
          <th>课程名称 (Cname)</th>
          <th>课程学分 (Ccredit)</th>
          <th>操作</th>
        </tr>
      </thead>
      <tbody>
        <tr v-for="(item, index) in items" :key="item.Cno">
          <td>{{ item.Cno }}</td>
          <td>{{ item.Cname }}</td>
          <td>{{ item.Ccredit }}</td>
          <td>
            <button @click="editItem(index)" class="action-btn edit-btn
">编辑</button>
            <button @click="deleteItem(item.Cno)" class="action-btn del
ete-btn">删除</button>
```

```
          </td>
        </tr>
      </tbody>
    </table>
    <!-- 弹窗 -->
    <div v-if="showForm" class="modal">
      <div class="modal-content">
        <h3>{{ formTitle }}</h3>
        <form @submit.prevent="submitForm" class="course-form">
          <div class="form-group">
            <label for="Cno">课程编号 (Cno):</label>
            <input id="Cno" v-model="formData.Cno" required />
          </div>
          <div class="form-group">
            <label for="Cname">课程名称 (Cname):</label>
            <input id="Cname" v-model="formData.Cname" required />
          </div>
          <div class="form-group">
            <label for="Ccredit">课程学分 (Ccredit):</label>
            <input id="Ccredit" v-model.number="formData.Ccredit" type="number" step="0.1" required />
          </div>
          <button type="submit" class="form-btn save-btn">{{ formButtonText }}</button>
          <button @click="cancelForm" type="button" class="form-btn cancel-btn">取消</button>
        </form>
      </div>
    </div>
    <!-- 模态背景 -->
    <div v-if="showForm" class="modal-overlay" @click="cancelForm"></div>
  </div>
</template>
<script>
import apiClient from '@/api/index'; // Axios 实例
export default {
  data() {
    return {
      items: [], // 用于存储课程数据
      showForm: false, // 控制弹窗显示
      formData: {
        Cno: '', // 课程编号
        Cname: '', // 课程名称
```

```
        Ccredit: null // 课程学分
      },
      editingIndex: null // 当前编辑的索引
    };
  },
  computed: {
    // 表单标题动态显示
    formTitle() {
      return this.editingIndex !== null ? '编辑课程' : '添加新课程';
    },
    formButtonText() {
      return this.editingIndex !== null ? '保存' : '添加';
    }
  },
  created() {
    // 页面加载时获取数据
    this.fetchItems();
  },
  methods: {
    // 从后端获取课程数据
    async fetchItems() {
      try {
        const response = await apiClient.get('/courses'); // 发起 GET 请
求
        this.items = response.data; // 将返回的数据赋值给 items
      } catch (error) {
        console.error('获取数据失败:', error); // 错误处理
      }
    },
    // 提交表单（添加或更新课程）
    async submitForm() {
      try {
        if (this.editingIndex !== null) {
          // 更新现有记录
          await apiClient.put(`/courses/${this.formData.Cno}`, this.for
mData);
        } else {
          // 添加新记录
          await apiClient.post('/courses', this.formData);
        }
        this.fetchItems(); // 提交后刷新列表
        this.resetForm(); // 重置表单
      } catch (error) {
        console.error('提交表单失败:', error);
```

```
      }
    },
    // 删除课程记录
    async deleteItem(Cno) {
      try {
        await apiClient.delete(`/courses/${Cno}`); // 根据课程编号删除
        this.fetchItems(); // 删除后刷新数据
      } catch (error) {
        console.error('删除记录失败:', error);
      }
    },
    // 编辑课程记录
    editItem(index) {
      this.editingIndex = index;
      this.formData = { ...this.items[index] }; // 将当前课程数据填充到表
单
      this.showForm = true;
    },
    // 取消表单操作
    cancelForm() {
      this.resetForm();
    },
    // 重置表单
    resetForm() {
      this.showForm = false;
      this.formData = { Cno: '', Cname: '', Ccredit: null };
      this.editingIndex = null;
    }
  }
};
</script>
<style scoped>
/* 表格样式 */
table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
  background-color: #f9f9f9;
  border: 1px solid #ccc;
}
th, td {
  padding: 10px;
  text-align: left;
  border: 1px solid #ccc;
```

```
}
/* 按钮样式 */
button {
  margin: 5px;
  padding: 8px 12px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
.add-btn {
  background-color: #4CAF50;
  color: white;
}
.action-btn {
  color: white;
}
.edit-btn {
  background-color: #2196F3;
}
.delete-btn {
  background-color: #f44336;
}
/* 弹窗样式 */
.modal {
  position: fixed;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.2);
  z-index: 1001;
  max-width: 500px;
  width: 100%;
}
.modal-content {
  display: flex;
  flex-direction: column;
}
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
```

```
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5);
  z-index: 1000;
}
/* 表单样式 */
.student-form .form-group {
  margin-bottom: 15px;
}
.student-form label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}
.student-form input, .student-form select {
  width: 100%;
  padding: 8px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}
/* 表单按钮样式 */
.form-btn {
  padding: 10px 15px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  margin-right: 10px;
}
.save-btn {
  background-color: #4CAF50;
  color: white;
}
.cancel-btn {
  background-color: #f44336;
  color: white;
}
</style>
```

（9）学生页面组件

```
<template>
  <div>
    <h2>学生管理</h2>
    <button @click="showForm = true" class="add-btn">添加新学生</button>
```

```
    <table>
      <thead>
        <tr>
          <th>学生编号 (Sno)</th>
          <th>学生姓名 (Sname)</th>
          <th>学生邮箱 (Semail)</th>
          <th>学生学分 (Scredit)</th>
          <th>学生宿舍 (Sroom)</th>
          <th>操作</th>
        </tr>
      </thead>
      <tbody>
        <tr v-for="(item, index) in items" :key="item.Sno">
          <td>{{ item.Sno }}</td>
          <td>{{ item.Sname }}</td>
          <td>{{ item.Semail }}</td>
          <td>{{ item.Scredit }}</td>
          <td>{{ item.Sroom }}</td>
          <td>
            <button @click="editItem(index)" class="action-btn edit-btn
">编辑</button>
            <button @click="deleteItem(item.Sno)" class="action-btn del
ete-btn">删除</button>
          </td>
        </tr>
      </tbody>
    </table>
    <!-- 弹窗 -->
    <div v-if="showForm" class="modal">
      <div class="modal-content">
        <h3>{{ formTitle }}</h3>
        <form @submit.prevent="submitForm" class="student-form">
          <div class="form-group">
            <label for="Sno">学生编号 (Sno):</label>
            <input id="Sno" v-model="formData.Sno" required />
          </div>
          <div class="form-group">
            <label for="Sname">学生姓名 (Sname):</label>
            <input id="Sname" v-model="formData.Sname" required />
          </div>
          <div class="form-group">
            <label for="Semail">学生邮箱 (Semail):</label>
            <input id="Semail" v-model="formData.Semail" />
          </div>
```

```
          <div class="form-group">
            <label for="Scredit">学生学分 (Scredit):</label>
            <input id="Scredit" v-model.number="formData.Scredit" type="number" step="0.1" required />
          </div>
          <div class="form-group">
            <label for="Sroom">学生宿舍 (Sroom):</label>
            <input id="Sroom" v-model="formData.Sroom" />
          </div>
          <button type="submit" class="form-btn save-btn">{{ formButtonText }}</button>
          <button @click="cancelForm" type="button" class="form-btn cancel-btn">取消</button>
        </form>
      </div>
    </div>
    <!-- 模态背景 -->
    <div v-if="showForm" class="modal-overlay" @click="cancelForm"></div>
  </div>
</template>
<script>
import apiClient from '@/api/index'; // Axios 实例，用于与后端通信
export default {
  data() {
    return {
      items: [], // 用于存储从数据库加载的学生数据
      showForm: false, // 控制弹窗显示
      formData: {
        Sno: '', // 学生编号
        Sname: '', // 学生姓名
        Semail: '', // 学生邮箱
        Scredit: null, // 学分
        Sroom: '' // 宿舍
      },
      editingIndex: null // 当前编辑的索引
    };
  },
  computed: {
    // 表单标题，根据是否为编辑状态动态显示
    formTitle() {
      return this.editingIndex !== null ? '编辑学生' : '添加新学生';
    },
    formButtonText() {
```

```
      return this.editingIndex !== null ? '保存' : '添加';
    }
  },
  created() {
    // 页面加载时获取数据
    this.fetchItems();
  },
  methods: {
    // 从后端获取学生数据
    async fetchItems() {
      try {
        const response = await apiClient.get('/students'); // 发
起 GET 请求
        this.items = response.data; // 将返回的数据赋值给 items
      } catch (error) {
        console.error('获取数据失败:', error); // 错误处理
      }
    },
    // 提交表单（添加或更新数据）
    async submitForm() {
      try {
        if (this.editingIndex !== null) {
          // 更新现有记录
          await apiClient.put(`/students/${this.formData.Sno}`, this.fo
rmData);
        } else {
          // 添加新记录
          await apiClient.post('/students', this.formData);
        }
        this.fetchItems(); // 提交后刷新列表
        this.resetForm(); // 重置表单
      } catch (error) {
        console.error('提交表单失败:', error);
      }
    },
    // 删除学生记录
    async deleteItem(Sno) {
      try {
        await apiClient.delete(`/students/${Sno}`); // 根据学生编号删除
        this.fetchItems(); // 删除后刷新数据
      } catch (error) {
        console.error('删除记录失败:', error);
      }
    },
```

```
    // 编辑学生记录
    editItem(index) {
      this.editingIndex = index;
      this.formData = { ...this.items[index] }; // 将当前学生数据填充到表
单
      this.showForm = true;
    },
    // 取消表单操作
    cancelForm() {
      this.resetForm();
    },
    // 重置表单
    resetForm() {
      this.showForm = false;
      this.formData = { Sno: '', Sname: '', Semail: '', Scredit: null,
Sroom: '' };
      this.editingIndex = null;
    }
  }
};
</script>
<style scoped>
/* 表格样式 */
table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
  background-color: #f9f9f9;
  border: 1px solid #ccc;
}
th, td {
  padding: 10px;
  text-align: left;
  border: 1px solid #ccc;
}
/* 按钮样式 */
button {
  margin: 5px;
  padding: 8px 12px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
.add-btn {
```

```
  background-color: #4CAF50;
  color: white;
}
.action-btn {
  color: white;
}
.edit-btn {
  background-color: #2196F3;
}
.delete-btn {
  background-color: #f44336;
}
/* 弹窗样式 */
.modal {
  position: fixed;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.2);
  z-index: 1001;
  max-width: 500px;
  width: 100%;
}
.modal-content {
  display: flex;
  flex-direction: column;
}
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5);
  z-index: 1000;
}
/* 表单样式 */
.student-form .form-group {
  margin-bottom: 15px;
}
.student-form label {
```

```
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}
.student-form input, .student-form select {
  width: 100%;
  padding: 8px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}
/* 表单按钮样式 */
.form-btn {
  padding: 10px 15px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  margin-right: 10px;
}
.save-btn {
  background-color: #4CAF50;
  color: white;
}
.cancel-btn {
  background-color: #f44336;
  color: white;
}
</style>
```

（10）教师页面组件

```
<template>
  <div>
    <h2>教师管理</h2>
    <button @click="showForm = true" class="add-btn">添加新教师</button>
    <table>
      <thead>
        <tr>
          <th>教师编号 (Tno)</th>
          <th>教师姓名 (Tname)</th>
          <th>教师邮箱 (Temail)</th>
          <th>教师薪水 (Tsalary)</th>
          <th>操作</th>
        </tr>
      </thead>
```

```
      <tbody>
        <tr v-for="(item, index) in items" :key="item.Tno">
          <td>{{ item.Tno }}</td>
          <td>{{ item.Tname }}</td>
          <td>{{ item.Temail }}</td>
          <td>{{ item.Tsalary }}</td>
          <td>
            <button @click="editItem(index)" class="action-btn edit-btn
">编辑</button>
            <button @click="deleteItem(item.Tno)" class="action-btn del
ete-btn">删除</button>
          </td>
        </tr>
      </tbody>
    </table>
    <!-- 弹窗 -->
    <div v-if="showForm" class="modal">
      <div class="modal-content">
        <h3>{{ formTitle }}</h3>
        <form @submit.prevent="submitForm" class="teacher-form">
          <div class="form-group">
            <label for="Tno">教师编号 (Tno):</label>
            <input id="Tno" v-model="formData.Tno" required />
          </div>
          <div class="form-group">
            <label for="Tname">教师姓名 (Tname):</label>
            <input id="Tname" v-model="formData.Tname" required />
          </div>
          <div class="form-group">
            <label for="Temail">教师邮箱 (Temail):</label>
            <input id="Temail" v-model="formData.Temail" />
          </div>
          <div class="form-group">
            <label for="Tsalary">教师薪水 (Tsalary):</label>
            <input id="Tsalary" v-model.number="formData.Tsalary" type=
"number" step="0.1" />
          </div>
          <button type="submit" class="form-btn save-btn">{{ formButton
Text }}</button>
          <button @click="cancelForm" type="button" class="form-btn can
cel-btn">取消</button>
        </form>
      </div>
    </div>
```

```
    <!-- 模态背景 -->
    <div v-if="showForm" class="modal-overlay" @click="cancelForm"></di
v>
  </div>
</template>
<script>
import apiClient from '@/api/index'; // 用于与后端通信的 Axios 实例
export default {
  data() {
    return {
      items: [], // 用于存储教师数据
      showForm: false,
      formData: {
        Tno: '',
        Tname: '',
        Temail: '',
        Tsalary: null
      },
      editingIndex: null
    };
  },
  computed: {
    formTitle() {
      return this.editingIndex !== null ? '编辑教师' : '添加新教师';
    },
    formButtonText() {
      return this.editingIndex !== null ? '保存' : '添加';
    }
  },
  created() {
    this.fetchItems(); // 页面加载时获取数据
  },
  methods: {
    async fetchItems() {
      try {
        const response = await apiClient.get('/teachers'); // 从后端获取
数据
        this.items = response.data; // 将数据赋值到 items
      } catch (error) {
        console.error('获取数据失败:', error);
      }
    },
    async submitForm() {
      try {
```

```
        if (this.editingIndex !== null) {
          // 更新现有记录
          await apiClient.put(`/teachers/${this.formData.Tno}`, this.fo
rmData);
        } else {
          // 添加新记录
          await apiClient.post('/teachers', this.formData);
        }
        this.fetchItems(); // 提交后刷新列表
        this.resetForm();
      } catch (error) {
        console.error('提交表单失败:', error);
      }
    },
    async deleteItem(Tno) {
      try {
        await apiClient.delete(`/teachers/${Tno}`);
        this.fetchItems(); // 删除后刷新列表
      } catch (error) {
        console.error('删除记录失败:', error);
      }
    },
    editItem(index) {
      this.editingIndex = index;
      this.formData = { ...this.items[index] }; // 填充表单数据
      this.showForm = true;
    },
    cancelForm() {
      this.resetForm();
    },
    resetForm() {
      this.showForm = false;
      this.formData = { Tno: '', Tname: '', Temail: '', Tsalary: null }
;
      this.editingIndex = null;
    }
  }
};
</script>
<style scoped>
/* 表格样式 */
table {
  width: 100%;
  border-collapse: collapse;
```

```
  margin-top: 20px;
  background-color: #f9f9f9;
  border: 1px solid #ccc;
}
th, td {
  padding: 10px;
  text-align: left;
  border: 1px solid #ccc;
}
/* 按钮样式 */
button {
  margin: 5px;
  padding: 8px 12px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}
.add-btn {
  background-color: #4CAF50;
  color: white;
}
.action-btn {
  color: white;
}
.edit-btn {
  background-color: #2196F3;
}
.delete-btn {
  background-color: #f44336;
}
/* 弹窗样式 */
.modal {
  position: fixed;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.2);
  z-index: 1001;
  max-width: 500px;
  width: 100%;
}
```

```css
.modal-content {
  display: flex;
  flex-direction: column;
}
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5);
  z-index: 1000;
}
/* 表单样式 */
.teacher-form .form-group {
  margin-bottom: 15px;
}
.teacher-form label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}
.teacher-form input {
  width: 100%;
  padding: 8px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}
/* 表单按钮样式 */
.form-btn {
  padding: 10px 15px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  margin-right: 10px;
}
.save-btn {
  background-color: #4CAF50;
  color: white;
}
.cancel-btn {
  background-color: #f44336;
  color: white;
```

```
}
</style>
```

（11）成绩页面组件

```
<template>
  <div>
    <h2>成绩管理</h2>
    <button @click="showForm = true" class="add-btn">添加新记录</button>
    <table>
      <thead>
        <tr>
          <th>学生编号 (Sno)</th>
          <th>教师编号 (Tno)</th>
          <th>课程编号 (Cno)</th>
          <th>分数 (Score)</th>
          <th>操作</th>
        </tr>
      </thead>
      <tbody>
        <tr v-for="(item, index) in items" :key="item.id">
          <td>{{ item.Sno }}</td>
          <td>{{ item.Tno }}</td>
          <td>{{ item.Cno }}</td>
          <td>{{ item.Score }}</td>
          <td>
            <button @click="editItem(index)" class="action-btn edit-btn
">编辑</button>
            <button @click="deleteItem(item.id)" class="action-btn dele
te-btn">删除</button>
          </td>
        </tr>
      </tbody>
    </table>
    <!-- 弹窗 -->
    <div v-if="showForm" class="modal">
      <div class="modal-content">
        <h3>{{ formTitle }}</h3>
        <form @submit.prevent="submitForm" class="student-form">
          <div class="form-group">
            <label for="Sno">学生编号 (Sno):</label>
            <input id="Sno" v-model="formData.Sno" required />
          </div>
          <div class="form-group">
            <label for="Tno">教师编号 (Tno):</label>
```

```
            <input id="Tno" v-model="formData.Tno" required />
          </div>
          <div class="form-group">
            <label for="Cno">课程编号 (Cno):</label>
            <input id="Cno" v-model="formData.Cno" required />
          </div>
          <div class="form-group">
            <label for="Score">分数 (Score):</label>
            <input id="Score" v-model.number="formData.Score" type="num
ber" step="0.1" required />
          </div>
          <button type="submit" class="form-btn save-btn">{{ formButton
Text }}</button>
          <button @click="cancelForm" type="button" class="form-btn can
cel-btn">取消</button>
        </form>
      </div>
    </div>
    <!-- 模态背景 -->
    <div v-if="showForm" class="modal-overlay" @click="cancelForm"></di
v>
  </div>
</template>
<script>
import apiClient from '@/api/index'; // 用于与后端通信的 Axios 实例
export default {
  data() {
    return {
      items: [], // 存储从数据库加载的成绩数据
      showForm: false, // 控制弹窗显示
      formData: {
        id: null,
        Sno: '', // 学生编号
        Tno: '', // 教师编号
        Cno: '', // 课程编号
        Score: null // 分数
      },
      editingIndex: null // 当前编辑的索引
    };
  },
  computed: {
    // 表单标题，根据是否为编辑状态动态显示
    formTitle() {
      return this.editingIndex !== null ? '编辑记录' : '添加新记录';
```

```
    },
    formButtonText() {
      return this.editingIndex !== null ? '保存' : '添加';
    }
  },
  created() {
    // 页面加载时获取数据
    this.fetchItems();
  },
  methods: {
    // 从后端获取成绩数据
    async fetchItems() {
      try {
        const response = await apiClient.get('/reports'); // 发起 GET 请
求
        this.items = response.data; // 将返回的数据赋值给 items
      } catch (error) {
        console.error('获取数据失败:', error); // 错误处理
      }
    },
    // 提交表单（添加或更新数据）
    async submitForm() {
      try {
        if (this.editingIndex !== null) {
          // 更新现有记录
          await apiClient.put(`/reports/${this.formData.id}`, this.form
Data);
        } else {
          // 添加新记录
          await apiClient.post('/reports', this.formData);
        }
        this.fetchItems(); // 提交后刷新列表
        this.resetForm(); // 重置表单
      } catch (error) {
        console.error('提交表单失败:', error);
      }
    },
    // 删除成绩记录
    async deleteItem(id) {
      try {
        await apiClient.delete(`/reports/${id}`); // 根据唯一 ID 删除
        this.fetchItems(); // 删除后刷新数据
      } catch (error) {
        console.error('删除记录失败:', error);
```

```
      }
    },
    // 编辑成绩记录
    editItem(index) {
      this.editingIndex = index;
      this.formData = { ...this.items[index] }; // 将当前成绩数据填充到表
单
      this.showForm = true;
    },
    // 取消表单操作
    cancelForm() {
      this.resetForm();
    },
    // 重置表单
    resetForm() {
      this.showForm = false;
      this.formData = { id: null, Sno: '', Tno: '', Cno: '', Score: nul
l };
      this.editingIndex = null;
    }
  }
};
</script>
<style scoped>
/* 表格样式 */
table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 20px;
  background-color: #f9f9f9;
  border: 1px solid #ccc;
}
th, td {
  padding: 10px;
  text-align: left;
  border: 1px solid #ccc;
}
/* 按钮样式 */
button {
  margin: 5px;
  padding: 8px 12px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
```

```css
}
.add-btn {
  background-color: #4CAF50;
  color: white;
}
.action-btn {
  color: white;
}
.edit-btn {
  background-color: #2196F3;
}
.delete-btn {
  background-color: #f44336;
}
/* 弹窗样式 */
.modal {
  position: fixed;
  top: 50%;
  left: 50%;
  transform: translate(-50%, -50%);
  background-color: #fff;
  padding: 20px;
  border-radius: 8px;
  box-shadow: 0px 4px 6px rgba(0, 0, 0, 0.2);
  z-index: 1001;
  max-width: 500px;
  width: 100%;
}
.modal-content {
  display: flex;
  flex-direction: column;
}
.modal-overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background: rgba(0, 0, 0, 0.5);
  z-index: 1000;
}
/* 表单样式 */
.student-form .form-group {
  margin-bottom: 15px;
```

```
}
.student-form label {
  display: block;
  margin-bottom: 5px;
  font-weight: bold;
}
.student-form input {
  width: 100%;
  padding: 8px;
  box-sizing: border-box;
  border: 1px solid #ccc;
  border-radius: 4px;
}
/* 表单按钮样式 */
.form-btn {
  padding: 10px 15px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
  margin-right: 10px;
}
.save-btn {
  background-color: #4CAF50;
  color: white;
}
.cancel-btn {
  background-color: #f44336;
  color: white;
}
</style>
```

（12）导航栏组件

```
<template>
  <div class="menu-container">
    <el-menu
      :default-active="activeIndex"
      class="el-menu-demo"
      mode="horizontal"
      @select="handleSelect"
      background-color="#545c64"
      text-color="#fff"
      active-text-color="#ffd04b">
      <el-menu-item index="1">学生</el-menu-item>
      <el-menu-item index="2">老师</el-menu-item>
```

```
      <el-menu-item index="3">课程</el-menu-item>
      <el-menu-item index="4">成绩</el-menu-item>
    </el-menu>
  </div>
</template>
<script>
import { Menu, MenuItem } from 'element-ui';
export default {
  components: {
    'el-menu': Menu,
    'el-menu-item': MenuItem
  },
  data() {
    return {
      activeIndex: '1' // 确保 activeIndex 与模板中的绑定一致
    };
  },
  methods: {
    handleSelect(key) {
      this.$emit('menu-select', key);
    }
  }
};
</script>
<style scoped>
.menu-container {
  width: 100%; /* 占据整个屏幕宽度 */
  margin: 0 auto; /* 水平居中 */
}
.el-menu-demo {
  background-color: #545c64;
  color: #fff;
  margin-bottom: 20px;
  display: flex; /* 使菜单项均匀分布 */
  justify-content: center; /* 中心对齐 */
}
.el-menu-demo .el-menu-item {
  flex: 1; /* 菜单项均分宽度 */
  text-align: center; /* 文本居中 */
  display: flex; /* 使用 Flex 布局 */
  align-items: center; /* 垂直居中 */
  justify-content: center; /* 水平居中 */
}
.el-menu-demo .el-menu-item.is-active {
```

```
  color: #ffd04b;
}
</style>
```

四、实验结论

在本次实验中，我们成功结合了 **Vue** 和 **Flask** 框架，并实现了与数据库的无缝连接，开发了一个功能完善的管理系统小程序。系统实现了增、删、改、查等基础功能，并采用前后端分离的开发模式，提升了开发效率。通过 Vue 的组件化设计，我们实现了数据表格和弹窗表单的动态渲染，极大地优化了用户的交互体验。而 Flask 后端则通过设计 RESTful API 接口，高效地处理了数据请求和数据库交互。

通过这次实验，我们进一步加深了对前后端分离开发模式的理解和实践能力。在前端开发中，我们掌握了 Vue 的核心功能，如数据绑定、事件处理和组件化思想；在后端开发中，通过 Flask 框架和 SQLAlchemy ORM 工具，熟悉了接口的设计与数据库的操作。此外，我们还解决了跨域请求、数据交互等实际问题，大幅提升了调试和问题定位的能力。这些经验为我们后续开发复杂系统奠定了坚实的基础。

尽管我们已经实现了系统的基本功能，但仍有优化的空间。例如，可以增加搜索、分页和权限管理等功能，以提升系统的实用性。同时，可以进一步优化界面美观度和用户体验，尝试将项目部署到线上环境。这次实验不仅完成了预期目标，也为我们积累了宝贵的实战经验，增强了开发复杂项目的信心和能力。